

Algorithm for file updates in Python

Project description

At my organization, access to restricted content is controlled with an allow list of IP addresses. The `"allow_list.txt"` file identifies these IP addresses. A separate remove list identifies IP addresses that should no longer have access to this content. I created an algorithm to automate updating the `"allow_list.txt"` file and remove these IP addresses that should no longer have access.

Open the file that contains the allow list

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"
# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]
# First line of `with` statement
with open(import_file, "r") as file:
```

I assigned the file name `"allow_list.txt"` to the variable `import_file`. I also created a list called `remove_list` containing the IP addresses that should no longer have access. Then I used a `with` statement along with the `open()` function to open `import_file` in read mode (`"r"`), storing the opened file in a variable called `file`. Using `with` ensures the file is automatically closed once I'm done working with it, even if an error occurs.

Read the file contents

```
with open(import_file, "r") as file:
    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()
```

When using an `.open()` function that includes the argument `"r"` for "read," I can call the `.read()` function in the body of the `with` statement. The `.read()` method converts the file into a string and allows me to read it. I applied the `.read()` method to the `file` variable identified in the `with` statement. Then, I assigned the string output of this method to the variable `ip_addresses`.

In summary, this code reads the contents of the `"allow_list.txt"` file into a string format that allows me to later use the string to organize and extract data in my Python program.

Convert the string into a list

```
# Use `.split()` to convert `ip_addresses` from a string to a list  
ip_addresses = ip_addresses.split()  
  
# Display `ip_addresses`  
print(ip_addresses)
```

The `.split()` method breaks up a string into individual elements and converts it into a list. I applied `.split()` to the `ip_addresses` variable, which was a string at this point. I assigned the list output of this method back to `ip_addresses`, overwriting the original string. Then, I used `print()` to display `ip_addresses` and confirm the conversion took place.

Iterate through the remove list

```
# Build iterative statement  
# Name loop variable `element`  
# Loop through `ip_addresses`  
  
for element in remove_list:
```

The `for` loop iterates through `remove_list`, using `element` as the loop variable to represent each IP address in `remove_list` as the loop runs. This sets up the iterative statement needed so that, in the next step, each IP address in `remove_list` can be checked against the allow list.

In summary, this code loops through each IP address in `remove_list`, one at a time, so I can compare them against the IP addresses in `ip_addresses`.

Remove IP addresses that are on the remove list

```
for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)
```

In order to remove IP addresses that are on the remove list, I used a conditional statement along with the `.remove()` method.

The conditional checks whether `element` (the current IP address from the loop) is in `remove_list`. If it is, the `.remove()` method is applied to `ip_addresses`, deleting that IP address from the list. This is possible without issues because `ip_addresses` doesn't contain any duplicate values, so `.remove()` won't accidentally leave a matching IP address behind.

Update the file with the revised list of IP addresses

In order to update the file with the revised list of IP addresses, I used the `.join()` method along with a `with` statement and the `.write()` method.

```
# Convert `ip_addresses` back to a string so that it can be written into the text file

ip_addresses = "\n".join(ip_addresses)
```

First, I used `.join()` to convert `ip_addresses` back into a single string, joining each IP address with `"\n"` so that every IP address appears on its own line. I assigned this string back to `ip_addresses`

```
# Build `with` statement to rewrite the original file

with open(import_file, "w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`

    file.write(ip_addresses)
```

Then, I built another `with` statement using the `open()` function to reopen `import_file`, this time in write mode (`"w"`), storing it in the variable `file`. Inside the `with` statement, I used the `.write()` method to overwrite the contents of the file with the updated `ip_addresses` string.

Summary

This algorithm updates an allow list by removing IP addresses that are no longer permitted to access restricted content. It opens `allow_list.txt` using a `with` statement and the `open()` function, then uses `.read()` to store its contents as a string in `ip_addresses`. The `.split()` method converts this string into a list, allowing individual IP addresses to be checked and removed. A `for` loop iterates through the IP addresses, and a conditional statement combined with the `.remove()` method deletes any IP address found in `remove_list`. Finally, `.join()` converts the updated list back into a string, and another `with` statement along with `.write()` saves the revised list back to the file.